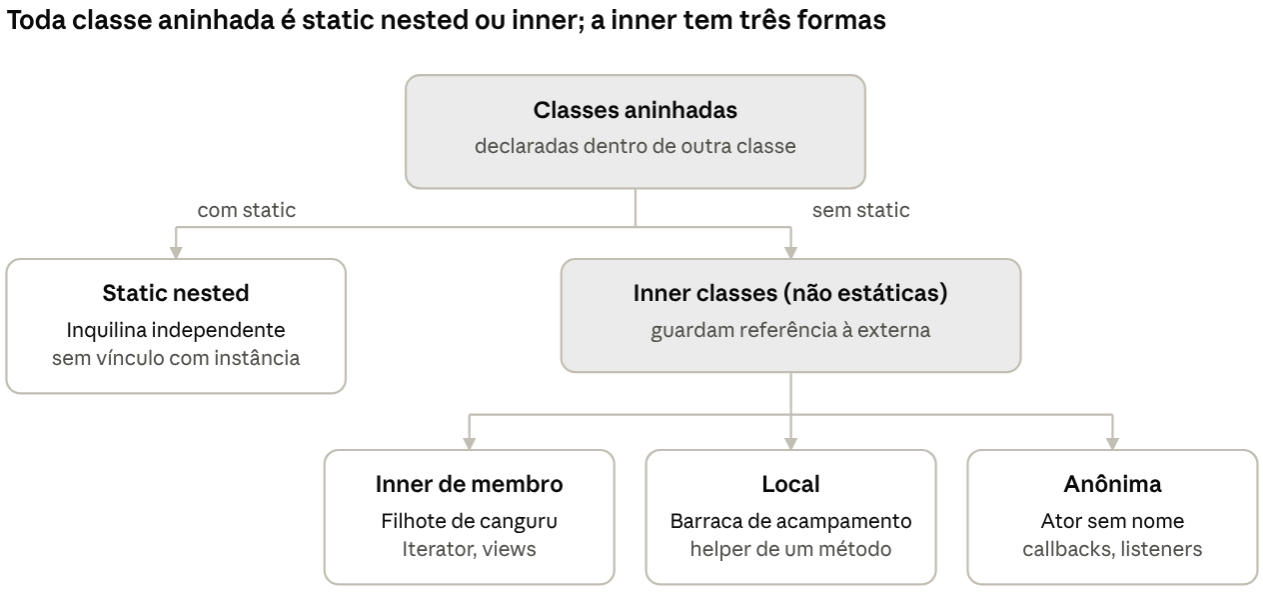
🪆 Guia Lúdico das Classes Aninhadas em Java

Oct 27, 2026 · @Jader

🌳 Árvore Genealógica — quem é quem na família



árvore genealógica das classes aninhadas · 4 tipos

A especificação do Java divide as classes declaradas dentro de outra em dois ramos: as **static nested**, que vivem por conta própria, e as **inner** (não estáticas), que carregam uma referência à instância externa. A diferença de uma palavra, `static`, decide quase todos os problemas descritos adiante.

🏠 Static Nested Class — a Inquilina Independente

Mora **dentro** da classe externa, mas paga o próprio aluguel: **não precisa de uma instância** da externa para existir. É basicamente uma classe normal que ganhou um sobrenome (`Externa.Interna`) e acesso aos membros `private static` da família.

Utilidade principal: agrupar uma classe que só faz sentido junto da externa, sem criar arquivo separado nem vínculo com objetos. Casos clássicos: *Builder*, nó de lista/árvore, DTOs de resposta, constantes agrupadas, `Map.Entry`.

Ela pode…	Ela não pode…
acessar membros `static` da externa, inclusive `private`	acessar campos/métodos **de instância** da externa diretamente
ter qualquer modificador (`public`, `private`, `protected`, pacote)	usar `Externa.this` (não existe "externa dona")
ter membros `static` e de instância	—

🧱 Exemplo 1 — Builder

```
public class Conta {
    private final String titular;
    private final double limite;

    private Conta(Builder b) {           // só o Builder constrói
        this.titular = b.titular;
        this.limite  = b.limite;
    }

    public static class Builder {        // 🏠 inquilina independente
        private String titular;
        private double limite = 500;

        public Builder titular(String t) { this.titular = t; return this; }
        public Builder limite(double l)  { this.limite = l;  return this; }
        public Conta build()             { return new Conta(this); }
    }
}

Conta c = new Conta.Builder()            // 👈 sem precisar de uma Conta
antes
        .titular("Jader")
        .limite(2_000)
        .build();
```

🔗 Exemplo 2 — Nó de lista encadeada

```
public class MinhaLista<T> {
    private Node<T> head;

    private static class Node<T> {       // detalhe interno, escondido do
mundo
        T valor;
        Node<T> proximo;
        Node(T valor) { this.valor = valor; }
    }

    public void addFirst(T v) {
        Node<T> n = new Node<>(v);
        n.proximo = head;
        head = n;
    }
}
```

> 💡 Por que `static` no `Node`? Cada nó não precisa conhecer a lista que o criou. Sem `static`, cada nó carregaria uma referência escondida para a lista — memória desperdiçada em cada objeto.

🦘 Inner Class (membro não estática) — o Filhote de Canguru

Vive **na bolsa** de um objeto da classe externa. Todo objeto inner carrega, escondida, uma referência para a instância externa que o criou (`Externa.this`). Por isso enxerga **tudo** da mãe: campos e métodos de instância, inclusive `private`.

Utilidade principal: criar um objeto auxiliar que trabalha *sobre o estado* de uma instância específica. Caso clássico: o `Iterator` de uma coleção, que precisa ler o array daquela coleção.

Ela pode…	Atenção…
ler e alterar campos de instância da externa, até `private`	só nasce a partir de uma instância: `externa.new Inner()`
chamar métodos da externa como se fossem dela	usa `Externa.this.x` para falar com a mãe quando há nomes repetidos
ter modificadores de acesso como qualquer membro	membros `static` só a partir do **Java 16** (antes: apenas constantes `static final`)

🔁 Exemplo — Iterator de uma pilha

```
public class Pilha implements Iterable<String> {
    private final String[] itens = new String[10];
    private int topo = 0;

    public void push(String s) { itens[topo++] = s; }

    @Override
    public Iterator<String> iterator() {
        return new PilhaIterator();          // nasce ligado a ESTA pilha
    }

    private class PilhaIterator implements Iterator<String> {   // 🦘 sem
static
        private int pos = topo;              // lê campo da mãe direto

        public boolean hasNext() { return pos > 0; }
        public String next()     { return itens[--pos]; }  // acessa array
private da mãe
    }
}
```

🤱 Criando um filhote de fora da classe

```
public class Banco {
    private String nome = "Banco Montanha";

    public class Agencia {
        private String numero;
        public Agencia(String numero) { this.numero = numero; }
        public String descricao() { return nome + " - ag. " + numero; }  //
usa campo da mãe
    }
}

Banco banco = new Banco();
Banco.Agencia ag = banco.new Agencia("0702");   // 👈 sintaxe esquisita, mas
é isso mesmo
ag.descricao();                                 // "Banco Montanha - ag. 0702"

// new Banco.Agencia("0702");                   // ❌ não compila: falta a mãe
```

🏕️ Classe Local — a Barraca de Acampamento

É montada **dentro de um método** (ou bloco) e desmontada quando ele termina: só existe ali dentro. Enxerga os membros da classe externa *e* as variáveis locais do método — desde que sejam `final` ou **effectively final** (nunca reatribuídas).

Utilidade principal: uma lógica auxiliar com nome, reutilizada algumas vezes dentro de um único método, que não merece virar membro da classe.

Regra	Detalhe
Escopo	visível só do ponto de declaração até o fim do bloco
Modificadores	não aceita `public`, `private`, `protected` nem `static`; aceita `final` ou `abstract`
Captura	variáveis locais e parâmetros precisam ser effectively final
Contexto	em método de instância, tem `Externa.this`; em método `static`, não tem
Primas modernas	Java 16+ permite *records*, *enums* e *interfaces* locais (sempre implicitamente static)

⛺ Exemplo — validador dentro de um método

```
public class Cadastro {

    public List<String> validar(String cpf, String email) {

        class Erro {                                    // ⛺ só existe aqui
dentro
            final String campo, motivo;
            Erro(String campo, String motivo) { this.campo = campo;
this.motivo = motivo; }
            @Override public String toString() { return campo + ": " +
motivo; }
        }

        List<Erro> erros = new ArrayList<>();
        if (cpf == null || cpf.length() != 11) erros.add(new Erro("cpf",
"deve ter 11 dígitos"));
        if (!email.contains("@"))            erros.add(new Erro("email",
"formato inválido"));

        return erros.stream().map(Erro::toString).toList();
    }
}
```

📌 Exemplo — capturando variável local

```
public Runnable saudacao(String nome) {
    String prefixo = "Olá, ";                // effectively final ✅

    class Saudador implements Runnable {
        public void run() { System.out.println(prefixo + nome); }
    }

    // prefixo = "Oi, ";                     // ❌ se descomentar, a classe
local não compila
    return new Saudador();
}
```

> 🧠 Por que a regra do effectively final? A classe local recebe uma **cópia** do valor. Se a variável pudesse mudar depois, a cópia e o original iriam divergir. O compilador proíbe para não haver surpresa.

🎭 Classe Anônima — o Ator Sem Nome

Entra em cena, faz **um único papel** e sai sem deixar nome nos créditos. É declarada e instanciada **na mesma expressão**, estendendo uma classe ou implementando uma interface.

Utilidade principal: fornecer uma implementação "descartável" no ponto de uso — *callbacks*, *listeners*, comparadores, sobrescrever um método de uma classe concreta.

Regra	Detalhe
Sintaxe	`new Tipo(args) { corpo }`
Herança	estende **uma** classe **ou** implementa **uma** interface, nunca as duas
Construtor	não pode declarar; usa bloco de inicialização `{ ... }` e passa args ao construtor da mãe
Captura	mesmas regras da classe local (effectively final)
`this`	aponta para o **objeto anônimo**, não para a classe externa

🎬 Exemplo 1 — implementando interface

```
Comparator<Cliente> porSaldo = new Comparator<>() {     // 🎭 (diamond em anônima: Java 9+)
    @Override
    public int compare(Cliente a, Cliente b) {
        return Double.compare(b.getSaldo(), a.getSaldo());  // maior saldo primeiro
    }
};
clientes.sort(porSaldo);
```

🎬 Exemplo 2 — estendendo classe e sobrescrevendo método

```
Thread worker = new Thread("processa-lote") {          // args vão para o
construtor de Thread
    @Override
    public void run() {
        System.out.println("Rodando em " + getName());
    }
};
worker.start();
```

⚖️ Anônima × Lambda

Se a interface tem **um único método abstrato** (interface funcional), a lambda faz o mesmo papel com menos texto:

```
clientes.sort((a, b) -> Double.compare(b.getSaldo(), a.getSaldo()));   // 🌟
lambda
clientes.sort(Comparator.comparingDouble(Cliente::getSaldo).reversed()); //
🌟 ainda melhor
```

Situação	Escolha
interface funcional, lógica curta	lambda ou method reference
interface com **vários** métodos a implementar	anônima
estender **classe** (abstrata ou concreta)	anônima
precisa de **estado próprio** (campos)	anônima (ou classe nomeada)
precisa que `this` seja o próprio objeto	anônima (na lambda, `this` é a classe externa)

🚨 Sala de Emergência — os principais problemas

1. 👻 Shadowing (sombreamento)

Quando um nome declarado num escopo **mais interno** é igual a um de escopo mais externo, o interno "faz sombra" e esconde o outro. O compilador não reclama — ele simplesmente usa o mais próximo. É a fonte de bugs mais sorrateira das classes aninhadas.

```
public class Externa {
    int x = 1;                                  // 🟦 nível 1: campo da
externa

    class Interna {
        int x = 2;                              // 🟩 nível 2: campo da
interna (sombra o 1)

        void mostrar(int x) {                   // 🟨 nível 3: parâmetro
(sombra o 2)
            System.out.println(x);              // 3  → o mais próximo vence
            System.out.println(this.x);         // 2  → campo da Interna
            System.out.println(Externa.this.x); // 1  → campo da Externa
        }
    }

    public static void main(String[] args) {
        new Externa().new Interna().mostrar(3);
    }
}
```

Quero acessar…	Escrevo
a variável local / parâmetro	`x`
o campo da classe onde estou	`this.x`
o campo da classe externa	`Externa.this.x`
um campo `static` da externa	`Externa.x`

Variantes do mesmo fantasma:

- 👻 **Método sombreado:** se a interna tem `salvar()` e a externa também, `salvar()` chama o da interna. Para o da externa: `Externa.this.salvar()`.
- 👻 **Herança + aninhamento:** uma anônima que estende uma classe com campo `nome` vai enxergar o `nome` **herdado** antes do `nome` da classe externa. Isso confunde muita gente.
- 👻 **Lambda é diferente:** lambda **não** cria escopo novo. Declarar um parâmetro com o mesmo nome de uma variável local é erro de compilação — na anônima, compila e sombreia.

```
String nome = "externo";
Runnable r1 = new Runnable() {
    public void run() { String nome = "anônima"; }   // ✅ compila (sombreia)
};
Consumer<String> r2 = nome -> {};                    // ❌ erro: 'nome' já
definido
```

2. 🎯 `this` ambíguo

Dentro de inner, local ou anônima, `this` é o **objeto aninhado**, não a externa. Em lambda, `this` é a externa. Passar `this` para um método esperando a externa causa erro de tipo — ou pior, compila e passa o objeto errado.

```
botao.addListener(new Listener() {
    public void clicou() {
        servico.registrar(this);              // ⚠️ passa o Listener anônimo
        servico.registrar(Tela.this);         // ✅ passa a Tela
    }
});
```

3. 🔒 "Variable must be final or effectively final"

Locais e anônimas só capturam variáveis locais que nunca são reatribuídas. Contador dentro de anônima? Não dá com `int` local.

```
int total = 0;
lista.forEach(new Consumer<>() {
    public void accept(Item i) { total += i.valor(); }   // ❌ não compila
});

// Saídas: AtomicInteger, campo da classe, ou (melhor) stream
int soma = lista.stream().mapToInt(Item::valor).sum();   // ✅
```

4. 💧 Vazamento de memória (o filhote que não larga a mãe)

Toda inner, local (em método de instância) e anônima (em contexto de instância) guarda referência oculta à externa. Se o objeto aninhado vive mais — num cache, numa thread longa, num listener registrado — a externa inteira **nunca é coletada** pelo GC.

```
public class Relatorio {
    private byte[] dados = new byte[50_000_000];      // 50 MB

    public Runnable tarefa() {
        return new Runnable() {                         // ⚠️ segura o
Relatorio inteiro
            public void run() { System.out.println("ok"); }
        };
    }
}
// Guardou a tarefa num executor de longa vida? Os 50 MB ficam presos junto.
```

> ✅ Cura: se não usa nada da instância externa, torne a classe `static` (ou use lambda que não capture `this`). Listeners: sempre remova ao descartar a tela.

5. 🍰 Double-brace initialization

```
List<String> l = new ArrayList<>() {{ add("a"); add("b"); }};   // ⚠️
```

Parece elegante, mas cria uma **subclasse anônima** a cada uso, captura a externa (vazamento) e quebra `equals` que compara `getClass()`. Prefira `List.of("a", "b")` ou `new ArrayList<>(List.of(...))`.

6. 📦 Serialização

Serializar inner, local ou anônima é **fortemente desaconselhado** pela especificação: o compilador gera campos sintéticos (como `this$0`) e nomes (`Externa$1`) que mudam entre compiladores e versões. Além disso, arrasta a externa junto. Para objetos serializáveis, use `static nested` ou classe de topo.

7. 🚫 Erros de instanciação e de contexto

Erro / sintoma	Causa	Correção
`non-static variable this cannot be referenced from a static context`	`new Inner()` dentro de `main` ou outro método `static`	criar a partir de uma instância (`externa.new Inner()`) ou tornar a classe `static`
`an enclosing instance that contains Externa.Inner is required`	`new Externa.Inner()` vindo de fora	`externa.new Inner()`

Erro / sintoma	Causa	Correção
`Illegal static declaration in inner class`	membro `static` em inner antes do Java 16	atualizar o Java, usar `static final` constante, ou mover para a externa
`non-static method cannot be referenced from a static context`	static nested tentando usar método de instância da externa	receber uma instância da externa como parâmetro

8. 🧩 Legibilidade

Anônima com 40 linhas, aninhamento de três níveis, classe local gigante: tudo compila, mas ninguém entende. O aninhamento existe para **esconder detalhes pequenos**, não para esconder sistemas inteiros.

🎒 Cola de Bolso — os quatro lado a lado

Característica	🏠 Static nested	🦘 Inner	🏕️ Local	🎭 Anônima
Onde é declarada	corpo da classe, com `static`	corpo da classe, sem `static`	dentro de método/bloco	numa expressão `new`
Tem nome?	sim	sim	sim	não
Precisa de instância externa?	não	sim	só se o método for de instância	só se o contexto for de instância
Acessa membros de instância da externa?	não (só via objeto recebido)	sim	sim (em método de instância)	sim (em contexto de instância)
Acessa variáveis locais?	—	—	sim, effectively final	sim, effectively final
Modificadores de acesso	todos	todos	nenhum	nenhum

Característica	🏠 Static nested	🦘 Inner	🏕️ Local	🎭 Anônima
Pode ter construtor?	sim	sim	sim	não (só bloco `{ }`)
Membros `static`	sim	Java 16+	Java 16+	Java 16+
Como instanciar	`new Externa.Nested()`	`externa.new Inner()`	`new Local()` dentro do bloco	no próprio ponto da declaração
Risco de vazamento	nenhum	alto	médio	alto
Arquivo `.class` gerado	`Externa$Nested`	`Externa$Inner`	`Externa$1Local`	`Externa$1`
Uso típico	Builder, Node, DTO, Entry	Iterator, views sobre o estado	helper de um só método	callback, listener, override pontual

🧭 Fluxograma de bolso

1. A classe só é usada **dentro de um método**? → se for uma vez só e for interface funcional: **lambda**; se for uma vez só e não for: **anônima**; se for várias vezes: **local**.
2. Ela precisa ler/alterar o **estado de uma instância** da externa? → **inner**.
3. Nenhum dos dois? → **static nested**.
4. Outras partes do sistema usam bastante? → provavelmente merece ser **classe de topo**, no próprio arquivo.

🏆 Boas Práticas — o Código de Conduta da Família

1. **Na dúvida, `static`.** 🏠 Se a classe aninhada não usa nada da instância externa, declare `static`. É a recomendação clássica do *Effective Java* (item "Favor static member classes over nonstatic"): sem referência oculta, sem vazamento, instanciação simples.
2. **Visibilidade mínima.** 🔐 Comece com `private`. Só abra (`public`) se fizer parte do contrato da classe, como um `Builder` ou um `Entry`.

3. **Lambda antes de anônima.** 🌟 Para interface funcional, prefira lambda ou *method reference*. Deixe a anônima para quando precisar estender classe, ter estado ou implementar vários métodos.
4. **Mantenha pequeno.** 📏 Uma anônima ou local com mais de ~10–15 linhas pede para virar classe nomeada (aninhada ou de topo).
5. **Evite shadowing de propósito.** 👻 Dê nomes diferentes aos campos da interna e da externa. Quando for inevitável, seja explícito com `this.x` e `Externa.this.x`. Ative o aviso de *hidden field* na IDE (IntelliJ, Eclipse e Checkstyle têm a regra `HiddenField`).
6. **Um nível de aninhamento basta.** 🪆 Classe dentro de classe dentro de classe vira matrioska — bonita na estante, ruim de manter.
7. **Cuide do ciclo de vida.** 💧 Inner e anônimas registradas como listener, guardadas em cache ou enviadas a threads longas devem ser removidas/canceladas quando a externa morrer. Ou use `static` + `WeakReference` quando precisar apontar para a externa sem segurá-la.
8. **Não serialize inner, local nem anônima.** 📦 Para o que vai para disco, fila ou rede, use `static nested`, `record` ou classe de topo.
9. **Esqueça o double-brace.** 🍰 Use `List.of`, `Map.of`, `Set.of` ou um *Builder*.
10. **Records aninhados para dados.** 💾 Em Java 16+, um `record` declarado dentro da classe é implicitamente `static` — ótimo para DTOs internos e para devolver vários valores de um método (o Java não tem tupla nativa; o record faz esse papel, com nomes nos campos):

```
public class ExtratoService {
    public record Linha(LocalDate data, String descricao, BigDecimal valor)
{}   // static implícito

    public List<Linha> gerar(Conta c) { /* ... */ }
}
```

11. **Teste pela pública.** 🧪 Classes aninhadas `private` são detalhe de implementação: teste o comportamento pela API da classe externa, não tente acessá-las por reflexão.

> 🎯 **Mantra final:** *aninhe para esconder detalhes, `static` para não carregar bagagem, lambda para ser breve, classe de topo quando crescer.*